

Министерство науки и высшего образования РФ
Федеральное государственное автономное образовательное учреждение высшего
образования “Омский государственный технический университет”
Кафедра “Автоматизированные системы обработки информации и управления”

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 1
по дисциплине «Кроссплатформенная разработка»
Войнов Роман Александрович ПИН-252т

Пояснительная записка
Шифр работы От-43-ПИН-252т-1 ЛР
Специальность 09.03.03 Программная инженерия

Преподаватель
Студент

М.А. Мищенко
Р.А. Войнов

Омск 2026

РЕФЕРАТ

Объектом выполненной работы является кроссплатформенное приложение «Каталог медицинских публикаций», разработанное на Kotlin с использованием Compose Multiplatform.

Целью данной работы является построение каркаса приложения: экран-списка публикаций с поиском на моковых данных, экрана публикации со связанными публикациями, навигации между экранами и локализации интерфейса.

В работе применены разделение кода на слои domain, data и ui, однонаправленный поток данных (состояние экрана и намерения пользователя), библиотека Navigation 3 и механизм строковых ресурсов Compose Multiplatform.

Степень внедрения — для учебных целей.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 ПОСТАНОВКА ЗАДАЧИ	5
2 АРХИТЕКТУРА ПРИЛОЖЕНИЯ	6
3 РЕАЛИЗАЦИЯ.....	7
3.1 Модель публикации и репозиторий.....	7
3.2 Моковые данные.....	7
3.3 Состояние экрана и намерения.....	7
3.4 Навигация	8
3.5 Связанные публикации	10
3.6 Тема.....	10
3.7 Локализация	10
4 ПРОВЕРКА РАБОТЫ	12
ЗАКЛЮЧЕНИЕ.....	16
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	17

ВВЕДЕНИЕ

Кроссплатформенная разработка позволяет описать интерфейс и логику приложения один раз и собрать их под несколько платформ. Технология Compose Multiplatform расширяет декларативный подход Jetpack Compose на настольные системы, веб и iOS: общий код лежит в одном модуле, а платформенные точки входа только запускают его.

Предметной областью приложения выбран каталог медицинских публикаций. Источником данных на следующих вехах станет программный интерфейс NCBI E-utilities базы PubMed. На первой вехе сеть не используется: приложение работает на моковых данных, а доступ к ним скрыт за интерфейсом репозитория, поэтому замена моков сетевым источником не затронет экраны.

Цель работы — разработать каркас кроссплатформенного приложения: экраны, навигацию между ними, локализацию интерфейса и разделение кода на слои.

Для достижения цели поставлены следующие задачи:

- создать проект Kotlin Multiplatform с таргетами Android, iOS, настольная система и веб;
- разложить код по слоям domain, data и ui;
- подготовить моковый набор из двадцати публикаций;
- реализовать экран-список с поиском и экран публикации со связанными публикациями;
- организовать навигацию на библиотеке Navigation 3;
- вынести цвета в тему, а подписи интерфейса — в строковые ресурсы на двух языках;
- включить отчёт компилятора Compose.

1 ПОСТАНОВКА ЗАДАЧИ

Приложение представляет собой каталог медицинских публикаций и состоит из двух экранов: списка публикаций и экрана публикации. Список показывает заголовки публикаций и поле поиска по заголовку. Экран публикации содержит категорию, заголовок, автора, описание и список связанных публикаций — того же автора или той же категории. Переход к связанной публикации открывает её экран поверх текущего.

Источником данных на следующих вехах выступает программный интерфейс NCBI E-utilities по адресу <https://eutils.ncbi.nlm.nih.gov/entrez/eutils/>. Для каталога нужны три метода: `esearch` возвращает идентификаторы публикаций (PMID) по запросу, `esummary` — краткие сведения о публикациях, `elink` — идентификаторы связанных публикаций. В запросах передаются параметры `tool` и `email`. Без ключа допускается три запроса в секунду, с ключом `api_key` — десять.

Требования к каркасу приведены в таблице 1.

Таблица 1 — Требования к каркасу приложения

Требование	Способ выполнения
Навигация на Navigation 3, маршрут — объект	Маршруты описаны sealed-интерфейсом Screen, идентификатор публикации лежит полем объекта
Возврат по одному экрану	Стек маршрутов — список, возврат снимает верхний маршрут; заданы все три спецификации переходов
Экран рисуется из состояния	В функцию экрана передаются состояние и обработчик намерений
Связанные публикации	На экране публикации выводятся публикации того же автора или той же категории
Подписи в строковых ресурсах, две локали	Каталоги <code>values</code> (английский) и <code>values-ru</code> (русский)
Тема из токенов оформления	Цвета объявлены только в файле темы, элементы читают их из цветовой схемы
Отчёт компилятора включён	Экраны остаются перезапускаемыми и пропускаемыми

2 АРХИТЕКТУРА ПРИЛОЖЕНИЯ

Код приложения разделён на три слоя. Слой `domain` содержит модель публикации и интерфейс репозитория. Слой `data` содержит моковые данные и реализацию репозитория. Слой `ui` содержит экраны, компоненты, тему, навигацию и модели представления. Зависимости направлены к слою `domain`: при подключении сети изменится только слой `data`.

Структура каталогов общего модуля приведена в листинге 1.

```
shared/src/commonMain/kotlin/com/example/medicineproject/  
  App.kt                корень: тема, репозиторий, навигатор, фабрики  
  Screen.kt             маршруты приложения  
  domain/               модель публикации и интерфейс репозитория  
  data/                 моки и реализация репозитория  
  list/ detail/         состояние, намерения, ViewModel и фабрики  
  ui/components/        переиспользуемые элементы интерфейса  
  ui/model/             модели представления  
  ui/navigation/        навигатор и хост навигации  
  ui/screens/           вёрстка экранов  
  ui/theme/             цветовая схема  
shared/src/commonMain/composeResources/  
  values/ values-ru/     подписи интерфейса на двух языках
```

Листинг 1 — Структура общего модуля

Логика каждого экрана вынесена в класс `ViewModel`. Состояние хранится в изменяемом потоке, а наружу отдаётся поток только для чтения. Действия пользователя приходят в единственный метод обработки намерений.

Репозиторий, навигатор и фабрики `ViewModel` создаются в корневой функции `App` и передаются вниз. Фабрики нужны потому, что `ViewModel` принимает зависимости в конструкторе, а в общем коде нет механизма, который создал бы её автоматически.

3 РЕАЛИЗАЦИЯ

3.1 Модель публикации и репозиторий

Модель публикации и интерфейс репозитория приведены в листинге 2. Функции репозитория приостанавливаемые: сейчас они отдадут моки сразу, а на следующей вехе будут выполнять сетевой запрос, не блокируя интерфейс.

```
data class Article(  
    val id: String,  
    val title: String,  
    val description: String,  
    val author: String,  
    val category: String,  
)  
  
interface ArticleRepository {  
    suspend fun getArticles(): List<Article>  
    suspend fun getArticleById(id: String): Article?  
}
```

Листинг 2 — Модель публикации и интерфейс репозитория

3.2 Моковые данные

Моки содержат двадцать публикаций восьми категорий. Фрагмент моков приведён в листинге 3.

```
val mockArticles = listOf(  
    Article(  
        id = "38123456",  
        title = "Genome analysis and evolutionary dynamics...",  
        description = "This study outlines modern...",  
        author = "Andrey L. S.",  
        category = "Science",  
    ),  
    ...  
)  
  
class ArticleRepositoryImpl : ArticleRepository {  
    override suspend fun getArticles(): List<Article> = mockArticles  
    override suspend fun getArticleById(id: String): Article? =  
        mockArticles.find { it.id == id }  
}
```

Листинг 3 — Фрагмент моковых данных и реализация репозитория

3.3 Состояние экрана и намерения

Экран не обрабатывает нажатия сам: ему передаются состояние и функция обработки намерений, решение принимает ViewModel. Если передать в экран ViewModel целиком, компилятор Compose признает параметр нестабильным и перестанет пропускать перерисовку экрана. Реализация экрана-списка приведена в листинге 4.

```

@Immutable
data class ArticleListState(
    val query: String = "",
    val articles: List<ArticleCardUi> = emptyList(),
    val isLoading: Boolean = true,
)

sealed interface ArticleListIntent {
    data class QueryChanged(val value: String) : ArticleListIntent
    data class ArticleClicked(val id: String) : ArticleListIntent
}

fun onIntent(intent: ArticleListIntent) {
    when (intent) {
        is ArticleListIntent.ArticleClicked ->
            navigator.open(Screen.ArticleDetail(intent.id))
        is ArticleListIntent.QueryChanged -> _state.update {
            it.copy(query = intent.value, articles = filter(intent.value))
        }
    }
}

```

Листинг 4 — Состояние, намерения и обработка намерений списка

Поиск фильтрует загруженный список по заголовку без учёта регистра. Список выводится ленивым столбцом, каждому элементу задан ключ по идентификатору публикации.

3.4 Навигация

В Navigation 3 стек маршрутов — обычный список объектов. Маршруты описаны sealed-интерфейсом, а идентификатор публикации лежит полем объекта, поэтому ошибку в аргументе находит компилятор. В маршрут кладётся только идентификатор, данные экран получает из репозитория.

```
sealed interface Screen {
    data object ArticleList : Screen
    data class ArticleDetail(val id: String) : Screen
}

class Navigator {
    private val _backStack =
        MutableStateFlow<List<Screen>>(listOf(Screen.ArticleList))
    val backStack: StateFlow<List<Screen>> = _backStack.asStateFlow()

    fun open(screen: Screen) {
        _backStack.update { it + screen }
    }

    fun back() {
        _backStack.update { stack ->
            if (stack.size > 1) stack.dropLast(1) else stack
        }
    }
}
```

Листинг 5 — Маршруты и навигатор

Хост навигации сопоставляет маршрутам экраны. Все три спецификации перехода заданы явно: значения по умолчанию для обратного и предиктивного переходов вне Android являются заглушками и приводят к ошибке во время выполнения.

```
NavDisplay(
    backStack = backStack,
    onBack = navigator::back,
    transitionSpec = { slide(SlideDirection.Start) },
    popTransitionSpec = { slide(SlideDirection.End) },
    predictivePopTransitionSpec = { slide(SlideDirection.End) },
    entryProvider = entryProvider {
        entry<Screen.ArticleList> { ... }
        entry<Screen.ArticleDetail> { key ->
            val viewModel: ArticleDetailViewModel = viewModel(
                key = "detail- $\{key.id\}$ ",
                factory = detailViewModelFactory,
                extras = ArticleDetailViewModelFactory.extrasFor(key.id),
            )
            ...
        }
    },
)
```

Листинг 6 — Хост навигации и создание ViewModel публикации

ViewModel экрана публикации получает ключ из идентификатора. Без ключа все публикации получили бы один экземпляр, и открытая связанная публикация показала бы данные предыдущей.

3.5 Связанные публикации

Связь публикации с другими записями каталога определяется полями `category` и `author`: связанными считаются публикации того же автора или той же категории. Вычисление вынесено в слой `domain`, приведено в листинге 7. При подключении NCBI связь будет приходить из метода `elink` списком PMID.

```
suspend fun ArticleRepository.getRelatedArticles(
    article: Article,
): List<Article> =
    getArticles().filter {
        it.id != article.id &&
        (it.author == article.author || it.category == article.category)
    }

is ArticleDetailIntent.RelatedClicked ->
    navigator.open(Screen.ArticleDetail(intent.id))
```

Листинг 7 — Поиск связанных публикаций и переход к ним

Переход к связанной публикации добавляет маршрут в стек, поэтому возврат закрывает экраны по одному в обратном порядке.

3.6 Тема

Цвета объявлены в одном файле темы и передаются в `MaterialTheme`. Элементы интерфейса читают цвета из `MaterialTheme.colorScheme`, а не задают их числом, поэтому оформление меняется в одном месте.

```
private val LightColorScheme = lightColorScheme(
    primary = Color(0xFF2D5482),
    onPrimary = Color.White,
    secondary = Color(0xFFD7DEEB),
    onSecondary = Color(0xFF2D5482),
    ...
)

@Composable
fun MedicineAppTheme(content: @Composable () -> Unit) {
    MaterialTheme(colorScheme = LightColorScheme, content = content)
}
```

Листинг 8 — Цветовая схема приложения

3.7 Локализация

Подписи интерфейса вынесены в строковые ресурсы. По ним создаётся класс `Res`, поэтому в коде используется сгенерированный идентификатор, и опечатка в имени строки не компилируется. Язык выбирается по локали системы.

```
<!-- values/strings.xml -->
<string name="related_articles">Related articles</string>
<string name="article_author">By %1$s</string>

<!-- values-ru/strings.xml -->
<string name="related_articles">Похожие статьи</string>
<string name="article_author">Автор: %1$s</string>

Text(stringResource(Res.string.article_author, article.author))
```

Листинг 9 — Строковые ресурсы и их использование

Локализуется интерфейс, а не данные каталога: заголовки, авторы и описания публикаций остаются на языке источника.

4 ПРОВЕРКА РАБОТЫ

Команды запуска и проверки приведены в листинге 10.

```
# настольная система
./gradlew :desktopApp:run
# настольная система, русская локаль
JAVA_TOOL_OPTIONS="-Duser.language=ru" ./gradlew :desktopApp:run
# Android
./gradlew :androidApp:assembleDebug
# отчёт компилятора Compose
./gradlew :shared:compileKotlinJvm
```

Листинг 10 — Команды запуска и проверки

Порядок проверки работы приведён в таблице 2.

Таблица 2 — Порядок проверки работы

Действие	Ожидаемый результат
Запустить приложение и прокрутить список	Двадцать карточек с заголовками публикаций
Ввести в поиск часть заголовка	Список фильтруется без учёта регистра; при отсутствии совпадений выводится сообщение «Ничего не найдено»
Открыть карточку	Экран публикации: категория, заголовок, автор, описание, связанные публикации
Открыть связанную публикацию и дважды нажать «Назад»	Экраны закрываются по одному: сначала возврат к первой публикации, затем к списку
Запустить приложение с русской локалью	Подписи интерфейса на русском языке, данные каталога на языке источника
Открыть отчёт компилятора Compose	Экраны помечены как перезапускаемые и пропускаемые

Отчёт компилятора Compose включён в сборке общего модуля и появляется в каталоге `shared/build/compose_compiler`. Выдержка из отчёта приведена в листинге 11.

```
restartable skipable fun ...ui.screens.list.ArticleListScreen(
    state: ArticleListState
    stable onIntent: Function1<ArticleListIntent, Unit>
    stable modifier: Modifier? = @static <expression>
)
restartable skipable fun ...ui.screens.detail.ArticleDetailScreen(
    state: ArticleDetailState
    stable onIntent: Function1<ArticleDetailIntent, Unit>
    stable modifier: Modifier? = @static <expression>
)
restartable skipable fun ...ui.components.ArticleCard(
    article: ArticleCardUi
    stable onClick: Function0<Unit>
    stable modifier: Modifier? = @static <expression>
)
```

Листинг 11 — Выдержка из отчёта компилятора Compose

Все три функции помечены как перезапускаемые и пропускаемые: Compose может вызвать функцию заново отдельно от родителя и не заходить в её тело, если параметры не изменились. Обработчики намерений и модификатор признаны стабильными, поэтому их передача не мешает пропуску перерисовки.

Внешний вид приложения приведён на рисунках 1–3.

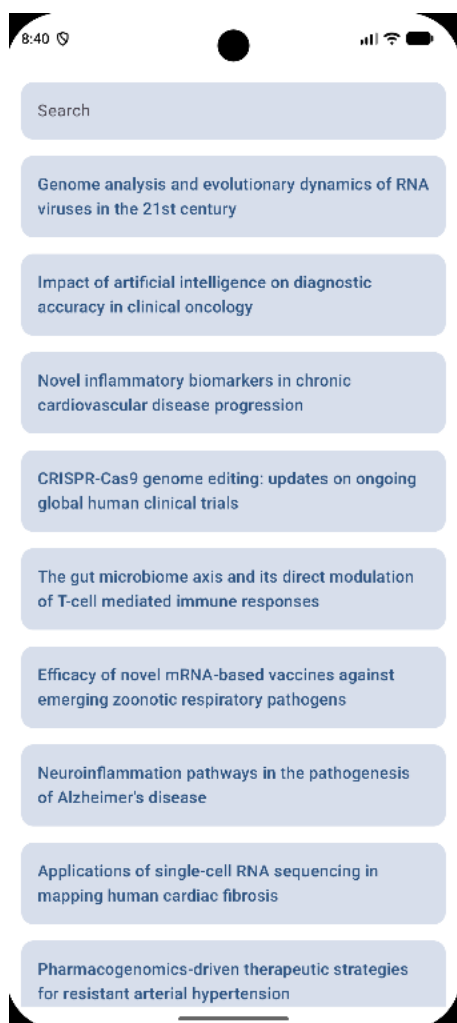


Рисунок 1 — Экран-список публикаций

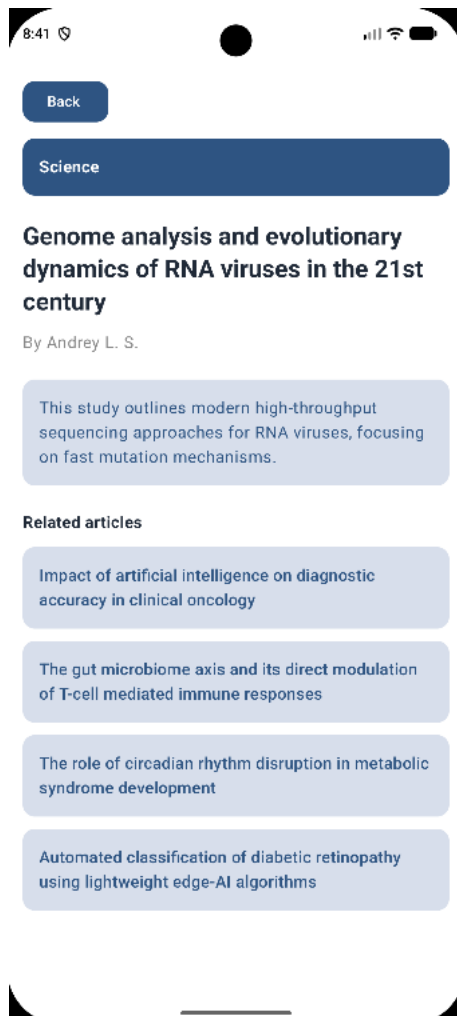


Рисунок 2 — Экран публикации

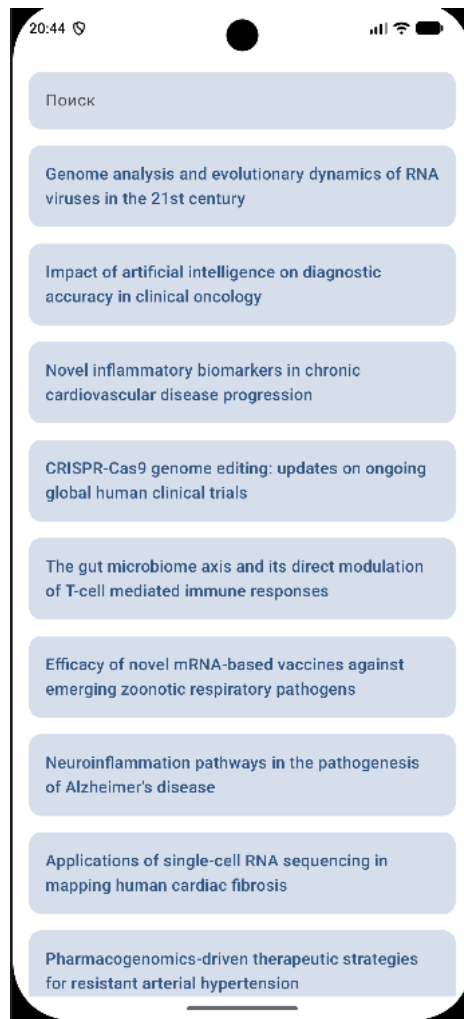


Рисунок 3 — Интерфейс на второй локали

ЗАКЛЮЧЕНИЕ

В результате выполнения лабораторной работы поставленная цель достигнута: разработан каркас кроссплатформенного приложения «Каталог медицинских публикаций» на Compose Multiplatform. Все поставленные задачи выполнены.

Код разложен по слоям domain, data и ui, данные получаются через интерфейс репозитория, поэтому замена моков на NCBI E-utilities затронет только слой data. Реализованы экран-список с поиском и экран публикации со связанными публикациями, навигация между ними построена на Navigation 3. Экраны получают состояние и обработчик намерений, а не ViewModel целиком. Цвета вынесены в тему, подписи интерфейса — в строковые ресурсы на английском и русском языках.

Полученный каркас служит основой для следующей вехи курса, на которой моки будут заменены запросами к E-utilities.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Compose Multiplatform: официальная документация [Электронный ресурс]. — URL: <https://www.jetbrains.com/help/kotlin-multiplatform-dev/compose-multiplatform.html>
2. Navigation 3: официальная документация [Электронный ресурс]. — URL: <https://developer.android.com/guide/navigation/navigation-3>
3. Entrez Programming Utilities Help [Электронный ресурс]. — URL: <https://www.ncbi.nlm.nih.gov/books/NBK25501/>
4. Material Design 3: цветовые схемы [Электронный ресурс]. — URL: <https://m3.material.io/styles/color/system>
5. Compose compiler reports [Электронный ресурс]. — URL: <https://developer.android.com/develop/ui/compose/performance/stability/diagnose>